

Implementação em Hardware de um Acelerador com Arranjo Sistólico para Multiplicação de Matrizes de Ordem N

Valmir Ferreira da Silva

Orientador: Prof. Gutemberg Gonçalves dos Santos Júnior, D.Sc.

Avaliador: Prof. Marcos Ricardo Alcantara Moraes, D.Sc.

Universidade Federal de Campina Grande – UFCG
Engenharia Elétrica

20 de maio de 2026



Motivação

- Multiplicação de matrizes aparece em **visão computacional, aprendizado profundo e processamento científico**.
- Em CPUs, a implementação clássica cresce com custo cúbico e explora pouco paralelismo espacial.
- Em aplicações intensivas, o gargalo passa a ser simultaneamente de **desempenho e eficiência energética**.

Plataforma FPGA

Reconfiguração + paralelismo em hardware

Estratégia Arranjo sistólico

Reuso local de dados e
Paralelismo Temporal e Espacial

Por que acelerar em hardware?

Limitações em software

- Execução sequencial do algoritmo;
- Maior pressão sobre memória e barramento;
- Crescimento rápido do tempo com o tamanho das matrizes.

Oportunidade com FPGA

- Paralelismo espacial explícito;
- Fluxo de dados controlado em hardware;
- Boa relação entre desempenho e flexibilidade.

Hipótese do trabalho

Um arranjo sistólico parametrizável em pode reduzir o tempo de cálculo para a multiplicação de matrizes, desde que o fluxo de entrada/saída e a comunicação com a FPGA sejam bem estruturados.

CPU → DMA/AXI → Systolic → CPU

Objetivo geral e contribuições

Objetivo geral

Projetar, implementar e validar um acelerador de multiplicação de matrizes baseado em *systolic array*, utilizando FPGA **Zynq UltraScale+ ZCU106**, para avaliar **desempenho, utilização de recursos, temporização e escalabilidade**.

Contribuições principais

- Arquitetura parametrizável $N \times N$;
- Integração PS–PL com DMA;
- Ambiente de verificação RTL e testes HiL;
- Avaliação experimental para múltiplos tamanhos de arranjo.

Objetivos específicos

Projeto e Integração

- Modelar o fluxo de dados;
- Projetar buffers, controle e interconexões;
- Descrever a arquitetura em nível RTL;
- Integrar comunicação entre software e hardware.

Validação e Análises

- Verificar funcionalmente com testbenches SystemVerilog;
- Executar simulação, síntese e implementação;
- Medir latência, throughput, recursos e potência;
- Comparar com implementação em CPU.

Metodologia do Trabalho

Modelagem → **RTL** → **Verificação** → **Integração PS-PL** →
Medições

Ferramentas

Xcelium, SimVision,
Vivado, Vitis e
VSCode

Hardware

Zynq UltraScale+
ZCU106

Interconexão

DMA + AXI Stream

Métricas

Latência, Timing,
Throughput e
Consumo

Multiplicação de matrizes

$$C = A \times B, \quad c_{ij} = \sum_{k=0}^{K-1} a_{ik} b_{kj}$$

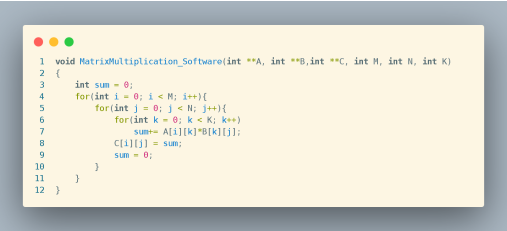
Cada elemento de C é obtido por uma sequência de produtos e acumulações entre uma linha de A e uma coluna de B .

- Para matrizes quadradas, o custo clássico cresce como $O(N^3)$.
- O padrão de cálculo é altamente regular e adequado a pipeline espacial.

Onde a oportunidade aparece?

- Muitos produtos independentes;
- Reuso de operandos entre resultados vizinhos;
- Estrutura repetida para diferentes valores de N .

Algoritmo clássico em software



```
1 void MatrixMultiplication_Software(int **A, int **B, int **C, int M, int N, int K)
2 {
3     int sum = 0;
4     for(int i = 0; i < M; i++){
5         for(int j = 0; j < N; j++){
6             for(int k = 0; k < K; k++){
7                 sum += A[i][k]*B[k][j];
8                 C[i][j] = sum;
9                 sum = 0;
10            }
11        }
12    }
```

Figura 1: Algoritmo clássico de multiplicação.

- Três laços aninhados percorrem linhas, colunas e produtos parciais;
- O paralelismo precisa ser extraído por compilador, vetorização ou múltiplos núcleos;
- O acesso frequente à memória torna-se relevante para matrizes maiores.

O algoritmo é simples de descrever, mas ineficiente para explorar **paralelismo maciço** de forma dedicada.

Conceito de arquitetura sistólica

- Arranjos sistólicos são redes regulares de *Processing Elements* (PEs);
- Os dados avançam ritmicamente, sincronizados pelo clock;
- Cada PE executa uma parcela pequena da computação e repassa dados aos vizinhos.

Reutilização de dados dentro da malha, com comunicação **local** em vez de acessos repetidos à memória externa.

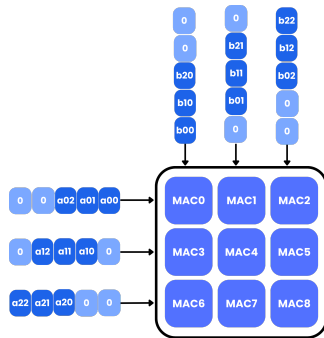
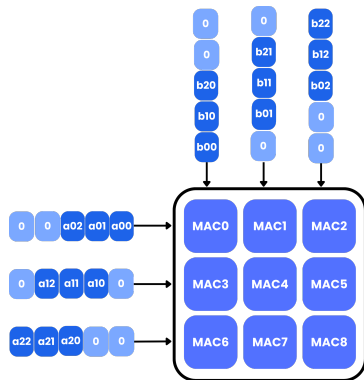


Figura 2: Fluxo bidimensional de operandos.

Fluxo de dados em um arranjo 3×3

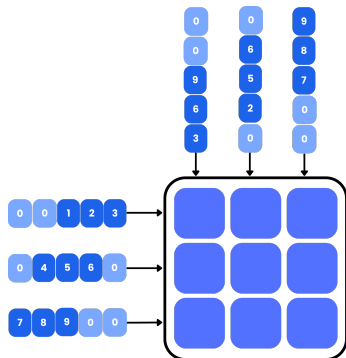


- Elementos de A avançam horizontalmente;
- Elementos de B avançam verticalmente;
- Cada PE calcula MACs e acumula um resultado parcial;

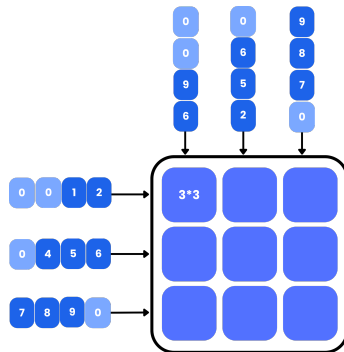
O desempenho depende tanto da computação local quanto do **alinhamento temporal** com que os dados entram no array.

Figura 3: Encontro entre elementos de A e B nos PEs.

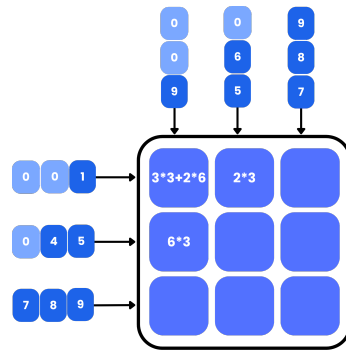
Janela temporal: Ciclos do 1º ao 3º



Ciclo 1: início do preenchimento

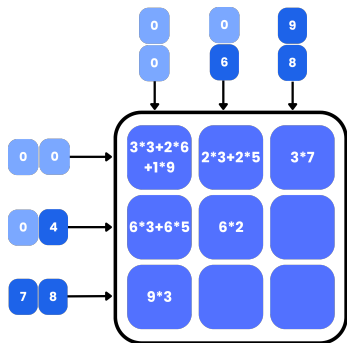


Ciclo 2: propagação dos primeiros operandos

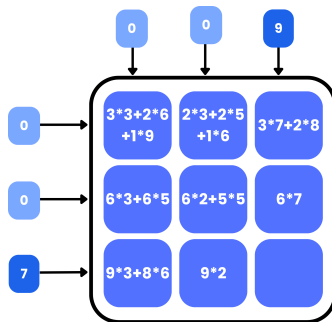


Ciclo 3: mais PEs passam a operar simultaneamente

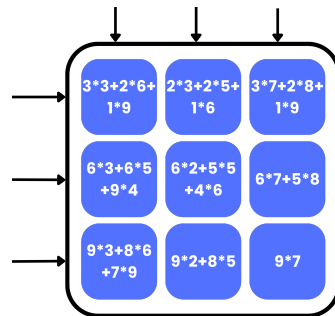
Janela temporal: Ciclos do 4º ao 6º



Ciclo 4: -



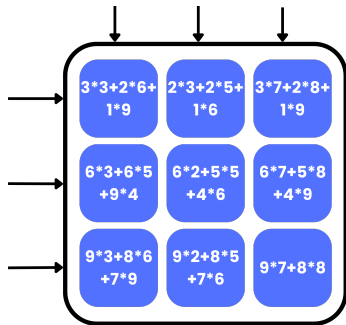
Ciclo 5: -



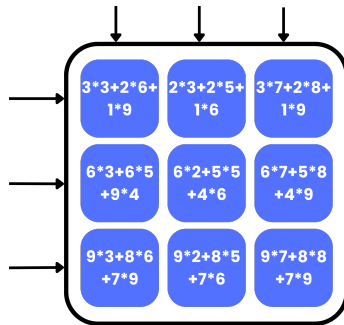
Ciclo 6: -

- A fase inicial é dominada pelo **enchimento do arranjo**.

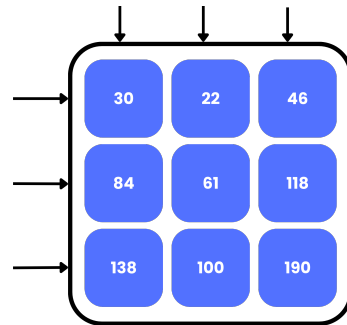
Janela temporal: Ciclos do 7º ao 8º



Ciclo 7: -

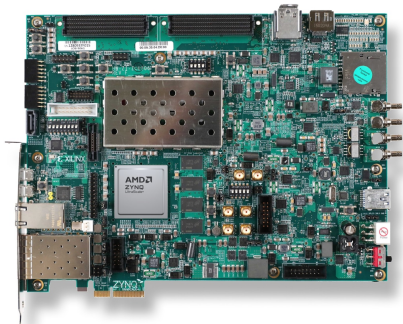


Ciclo 8: -



Resultado

Plataforma Experimental



- **PS:** processador ARM Cortex-A53 para coordenação e comunicação.
- **PL:** implementação do acelerador sistólico e buffers internos.
- **Fluxo de teste:** PS envio via DMA, processamento no array e coleta de resultados.

A plataforma permite estudar o sistema completo: **Algoritmo + Hardware + Comunicação.**

Evolução do sistema ao longo do projeto

Ver.	Plataforma	Comunicação	Gargalo dominante	Melhoria introduzida
0	ARTY	UART	baixa largura de banda e baixa escalabilidade	validação funcional inicial
1	ZYNQ	Ethernet + FIFO	pré-processamento das matrizes no host	migração para PS+PL
2	ZYNQ	Ethernet + FIFO	envio de dados já deslocados	deslocamento incorporado ao hardware
3	ZYNQ	DMA + AXI Stream	overhead de movimentação de dados	transferência por blocos e menor latência

O ganho de desempenho não veio apenas do arranjo sistólico; ele dependeu fortemente da **evolução da infraestrutura de comunicação**.

Latência do arranjo e implicações

$$L_{\text{arranjo}} = (2N - 1) + N$$

- $(2N - 1)$: preenchimento e deslocamento dos dados no array;
- N : tempo adicional para escoar a saída.

O ganho do array cresce com o paralelismo, mas a alimentação e a drenagem de dados continuam sendo partes fundamentais da latência total.

Escalabilidade

Replicação

mais PEs aumentam paralelismo

Risco

Timing

interconexão cresce com N

Risco

Recursos

uso de LUTs e CLBs cresce quase quadraticamente

Integração com o Zynq e DMA/AXI Stream

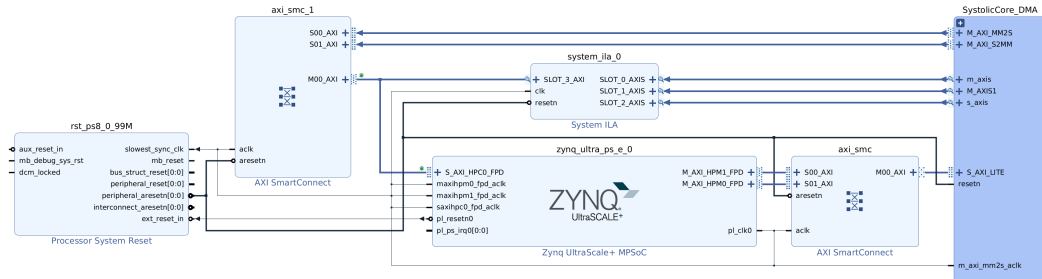


Figura 5: Integração do acelerador ao subsistema PS-PL.

- O DMA transfere blocos de dados entre memória do sistema e a lógica programável.

Integração com o Zynq e DMA/AXI Stream

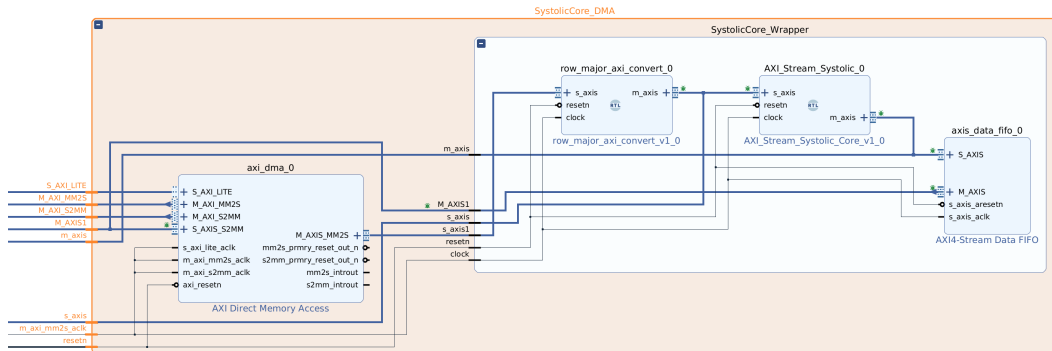


Figura 6: Integração do acelerador ao subsistema PS-PL.

Arquitetura Geral do Acelerador

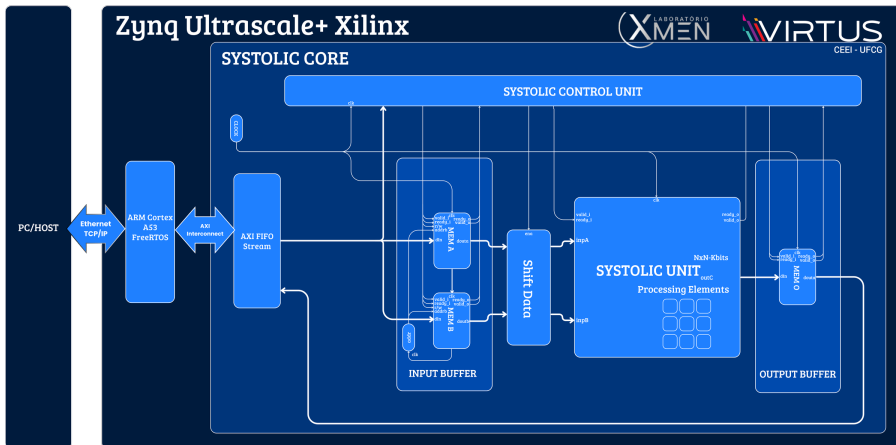


Figura 7: Diagrama em blocos da Arquitetura Proposta.

Estratégia de fluxo de dados em *streaming*

- O host/PS envia operandos ao acelerador por DMA/AXI Stream;
- Um FIFO absorve variações de latência e desacopla produtor e consumidor;
- Buffers independentes armazenam temporariamente matrizes A e B ;
- O bloco de deslocamento reorganiza os dados no tempo correto para o array.

Entrada

por linhas

Menor overhead que envio elemento a elemento

Computação

streaming

Alimentação contínua do array

Saída

por linhas

Compatível com a taxa do núcleo

Parametrização do arranjo sistólico

Parâmetros principais

- **SIZE**: dimensão do arranjo $N \times N$;
- **WIDTH**: largura dos operandos de entrada;

Objetivo

A mesma arquitetura-base pode ser reusada para diferentes tamanhos de problema, permitindo estudar escalabilidade.

Consequência direta Aumentar N eleva o número de PEs para N^2 , o que favorece paralelismo, mas também aumenta demanda por interconexão, recursos lógicos e margem temporal.

Fluxo de Dados e Execução do Acelerador

→ Carregar A → Carregar B → Processar no array → Transmitir →

Controle determinístico

FSMs definem quando cada módulo pode avançar.

Comunicação robusta

DMA/AXI e handshake validam cada transferência.

Estratégia de verificação funcional

- testbenches em SystemVerilog aplicaram vetores de teste representativos;
- foram usados valores aleatórios e padrões específicos para exercitar caminhos distintos;
- as saídas do hardware foram comparadas com uma implementação de referência em software.

Resultado principal

As simulações confirmaram correspondência funcional entre a arquitetura RTL e o modelo de referência para os casos avaliados.

Tamanhos simulados
2 a 128

com 4 bits de entrada e 8 bits de saída

Limite prático
256+

tempo de simulação/síntese elevado

Ambiente de verificação funcional do DUT

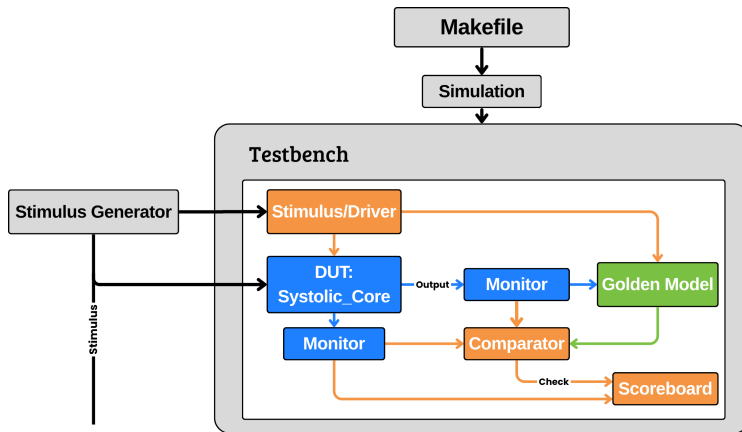
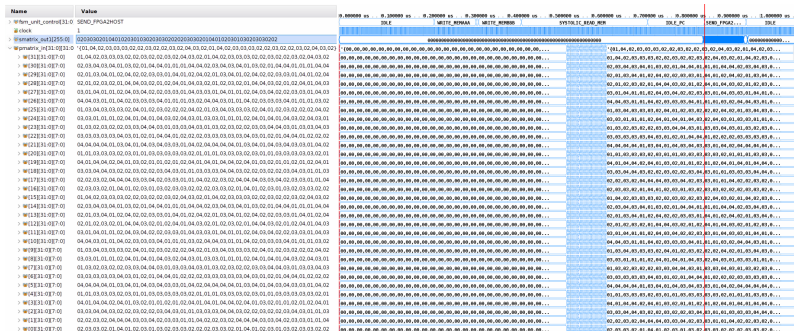


Figura 8: Ambiente de verificação do acelerador sistólico.

Exemplo de simulação funcional: caso 32×32 Figura 9: Forma de onda de uma multiplicação 32×32 com $I \times B = B$.

- As formas de onda evidenciam sincronização adequada entre comunicação, controle e processamento interno.

Latência por etapa e por versão

Versão	Escrita A/B	Array	Saída
1.0	$2N(2N - 1)$	$3N$	N^2
2.0	$2N^2$	$3N$	N^2
3.0	$2N$	$3N$	N

A versão otimizada reduz drasticamente o peso da comunicação, que sai de termos quadráticos para um comportamento linear em N .

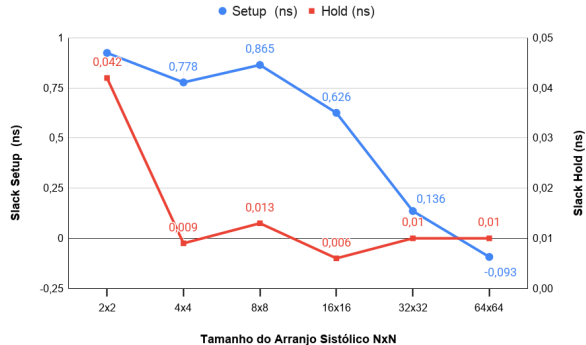
Efeito da otimização arquitetural na complexidade

Latência Total:

- versão 1.0: $5N^2 + N + 15$;
- versão 2.0: $3N^2 + 3N + 15$;
- versão 3.0: $6N + 15$.

Ao deslocar funções de preparação para o hardware e transmitir dados em blocos, o sistema deixa de ser dominado por movimentação quadrática e passa a se aproximar do comportamento do próprio arranjo sistólico.

Análise de temporização a 333 MHz

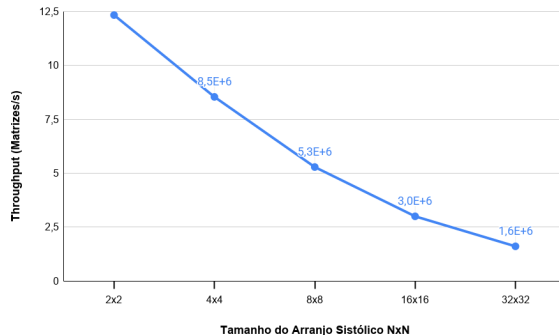


- até 16×16 , o *setup slack* permanece positivo;
- o melhor valor aparece próximo de 8×8 ;
- em 64×64 , há violação de *setup* ($-0,093$ ns).

Escalar o array aumenta o caminho crítico e dificulta o fechamento temporal.

Figura 10: Slack de setup e hold em função de N .

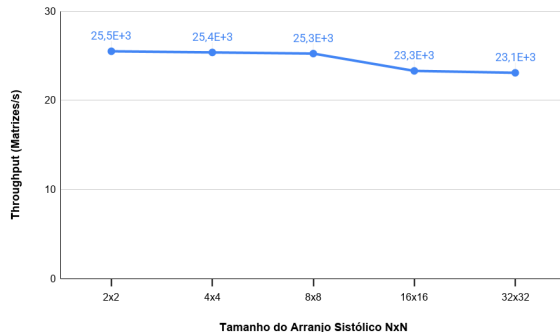
Throughput teórico do acelerador



- mede a capacidade máxima do hardware sem penalidades de comunicação;
- arranjos menores processam mais matrizes completas por segundo devido ao menor custo de preenchimento/escoamento.

Figura 11: Throughput ideal em Mmatrizes/s.

Throughput medurado com DMA



- o valor medido é bem menor que o teórico;
- o custo de iniciar transações e mover dados passa a dominar o tempo total, sobretudo para matrizes pequenas.

Figura 12: Throughput observado em kmatrizes/s.

Teórico x Mensurado

- Limitante não é a capacidade de MAC;
- **DMA/AXI** reduz throughput efetivo;
- Lote + *buffering* + *tiling* → próximo do ideal.

Hardware ideal

Mmatrizes/s

limite do núcleo

Sistema real

kmatrizes/s

com transferência

Melhoria

Lote/BRAM

amortizar DMA

Utilização absoluta de recursos da FPGA

- o crescimento acompanha aproximadamente N^2 ;
- cada aumento de N replica mais PEs e amplia interconexões.

Exemplo extremo:
 64×64 : cerca de 174 mil LUTs e 27,5 mil CLBs.

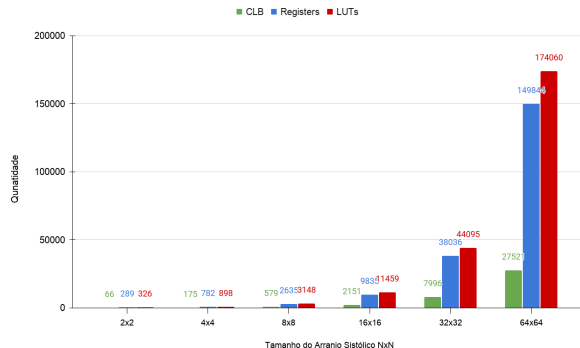


Figura 13: LUTs, registradores e CLBs por configuração.

Utilização percentual e limite da plataforma

- até 32×32 , a implementação ainda é confortável;
- em 64×64 , a ocupação chega a **76,1%** das LUTs e **96,1%** dos CLBs.

A arquitetura escala, mas a plataforma escolhida impõe um teto físico claro.

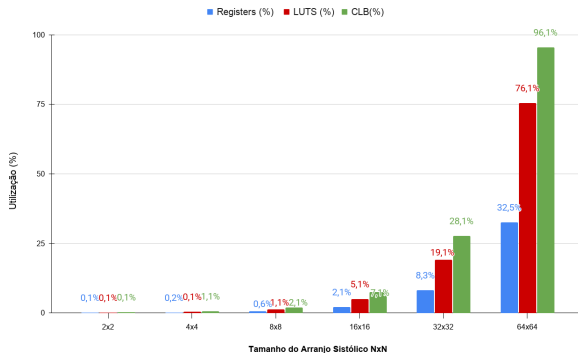


Figura 14: Ocupação percentual dos recursos do FPGA.

Potência consumida e decomposição interna

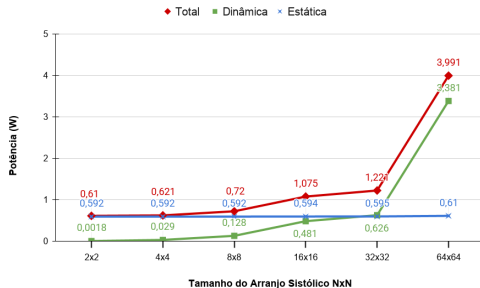


Figura 15: Potência total, dinâmica e estática.

- para 64×64 , a potência total chega a **3,991 W**, com dominância da parcela dinâmica.

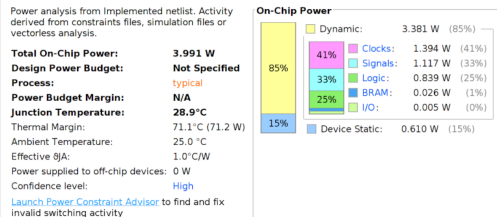
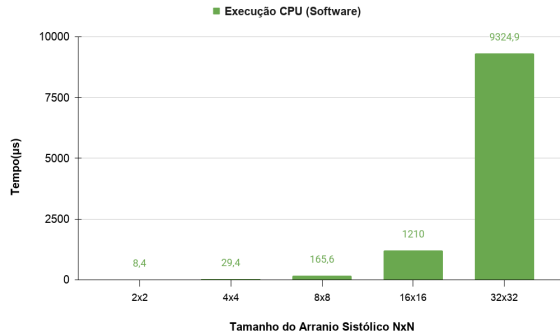


Figura 16: Decomposição da potência para 64×64 .

Referência em software no ARM Cortex-A53 @ 1.2 GHz



- o tempo cresce rapidamente com N ;
- de $8,4 \mu s$ em 2×2 até $9314,3 \mu s$ em 32×32 ;
- o comportamento é coerente com custo cúbico do algoritmo clássico.

Figura 17: Tempo da implementação em software.

Hardware medido vs. execução teórica do array

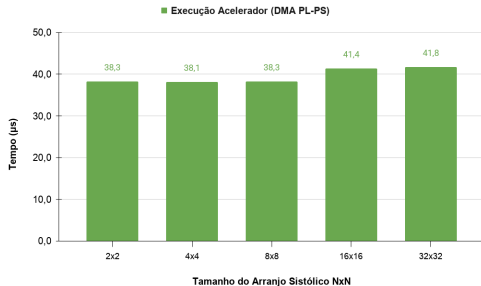


Figura 18: Tempo medido do sistema PS+PL.

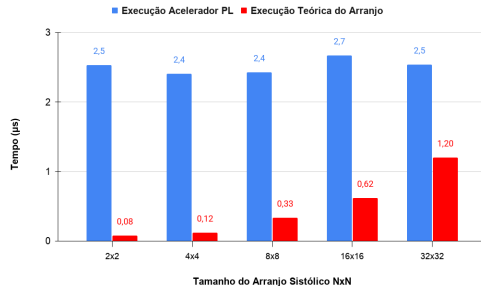
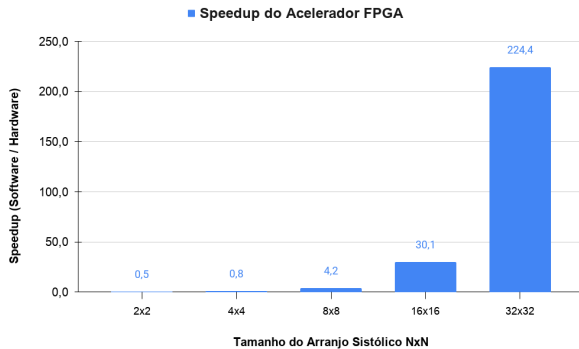


Figura 19: Tempo medido vs. ideal do arranjo.

- o hardware permanece em faixa de dezenas de microssegundos, mas ainda acima do limite teórico devido ao custo de comunicação.

Speedup do acelerador FPGA



- 2×2 : ganho ainda baixo ($0,5\times$);
- 8×8 : cerca de $4,2\times$;
- 16×16 : cerca de $30,1\times$;
- 32×32 : acima de $200\times$.

Figura 20: Ganho em relação à implementação em software.

Execução no ambiente de sistemas embarcados da Xilinx/AMD

- Tipo do teste: $\alpha \cdot I \cdot B = B$;

```
--- Exiting main() ---
PS-PL
Tempo software: 9315   us | Ciclos:   93144
Tempo Envio:   28     us | Ciclos:   285
Tempo Recepcao: 17    us | Ciclos:   178
Tempo Total:   45     us | Ciclos:   463
9 3 1 1 4 6 8 5 2 1 7 3 3 5 6 4 0 4 0 6 1 5 8 4 1 8 8 9 4 4 8 7
0 7 1 6 4 1 8 8 4 7 5 4 1 3 3 7 2 4 7 5 4 6 0 4 0 9 7 0 3 5 2 4
7 8 6 8 8 8 1 7 7 8 3 0 0 5 4 7 7 8 6 9 2 4 6 3 4 7 1 0 8 1 4 4
4 4 6 2 6 3 7 0 6 7 5 1 4 9 3 7 3 3 8 3 4 3 3 0 1 1 8 8 3 4 7 8
9 6 5 2 3 7 2 0 2 5 3 8 7 1 4 0 4 1 6 9 3 8 6 0 2 4 3 0 5 0 5 2
5 5 1 4 0 8 9 9 2 4 6 8 2 4 2 1 0 5 5 7 9 1 0 6 6 4 8 2 3 2 5
4 1 1 1 6 6 3 2 8 2 6 8 6 5 8 2 9 7 6 4 2 2 7 7 4 8 3 2 6 3 1 9
4 3 4 9 3 7 9 5 4 1 0 1 5 1 7 6 8 3 6 2 7 0 2 3 5 9 1 0 8 3 4 8
3 8 8 7 3 8 7 9 0 1 0 8 2 7 3 1 7 2 9 4 7 0 2 7 8 5 4 7 8 7 4 9
9 3 8 7 0 5 4 6 8 9 8 5 3 3 0 1 3 9 7 0 1 5 4 0 7 8 6 1 7 6 0 0
7 7 1 0 7 9 6 9 0 6 6 4 4 4 8 5 1 0 8 0 1 6 0 4 9 7 7 2 2 4 9 9
9 4 5 7 9 8 8 4 6 5 5 9 7 1 5 9 9 3 1 3 4 7 7 1 2 4 0 0 2 7 5 9
0 2 8 1 2 7 4 8 9 4 6 0 3 1 6 3 3 3 6 1 5 5 1 1 5 5 0 4 3 4 9 4
5 8 6 7 1 0 3 0 3 2 2 5 4 0 0 9 1 2 6 5 5 1 6 4 6 4 8 9 1 8 6 3
5 8 2 3 4 8 2 0 5 2 4 4 5 6 6 9 4 6 3 1 9 5 2 1 3 8 0 7 1 2 3 0
7 5 9 2 3 3 2 2 4 3 5 6 7 1 8 3 2 9 1 4 0 6 1 0 1 8 9 8 4 5 8 4
7 6 5 1 7 3 5 4 4 0 6 0 9 5 1 9 9 8 2 0 3 3 8 0 3 0 2 5 4 2 5 5
1 6 8 5 3 5 6 7 3 0 7 1 5 3 9 7 3 9 1 0 9 4 1 3 2 1 2 1 8 2 6 7
2 0 3 3 8 5 5 9 0 1 7 5 4 7 0 5 9 1 9 7 0 0 6 8 0 7 5 6 8 7 6 2
2 6 7 2 3 3 7 4 5 0 8 4 5 0 5 8 9 6 7 3 6 6 3 7 6 1 8 6 6 2 2 3
8 1 1 2 8 9 8 3 1 7 0 0 6 9 0 3 5 9 1 0 0 6 9 7 1 8 4 0 8 6 5 0
9 3 7 8 6 4 6 0 0 3 4 2 4 5 9 3 2 2 6 7 0 3 2 9 5 6 0 9 8 9 1 6
7 6 2 0 9 0 4 3 2 7 7 1 8 4 3 9 6 6 9 5 8 1 0 6 2 7 7 5 5 8 9 5
0 4 2 8 5 5 3 0 6 9 2 9 0 5 7 4 9 7 2 6 6 1 0 6 1 8 3 6 5 0 7 3
3 3 6 8 5 7 0 9 6 6 5 1 7 5 9 6 7 2 6 4 2 5 2 5 9 4 9 8 9 9 3 5
1 0 7 0 6 5 5 3 2 8 4 6 7 1 7 3 5 5 5 9 7 1 5 7 9 5 9 4 9 5 7 8
2 7 4 3 8 5 9 9 0 1 5 6 1 6 3 9 9 3 2 6 4 2 8 9 5 1 0 5 0 8 0 1
1 9 7 7 1 1 9 7 5 0 8 8 0 7 2 6 7 5 3 6 3 4 1 1 2 9 9 9 8 2 1 0
5 5 1 6 1 7 3 0 1 7 6 7 7 7 3 2 3 6 3 4 4 3 3 7 8 5 9 9 8 8 0 6
1 8 7 1 0 2 8 2 8 4 0 1 1 2 1 3 7 4 3 8 8 4 4 5 9 1 9 7 5 1 6
5 2 1 2 7 1 2 0 1 2 4 6 9 8 0 9 4 3 1 6 5 8 9 0 6 8 4 2 9 9 1 9
6 4 6 1 9 7 9 1 0 5 9 5 7 1 7 0 4 3 8 7 6 1 4 3 0 4 1 4 7 1 5 8
Successfully ran XAxisDma_SimplePoll Example
```

Figura 21: Ambiente de sistemas embarcados da Xilinx/AMD

Trade-offs de Escalabilidade

Benefício

Paralelismo

mais PEs aceleram o
cálculo

Custo

Recursos

LUTs, CLBs e
interconexão crescem
rápido

Risco

Energia e timing

clock e roteamento
tornam-se críticos

- O melhor ponto de operação depende da meta do sistema: throughput máximo, consumo, área ou frequência-alvo.

Resumo de Resultados

Arranjo	Setup	Potência total	Uso de LUTs	Observação
2×2	positivo	$\sim 0,61$ W	0,1%	baixa carga computacional
16×16	positivo	$\sim 1,08$ W	5,1%	bom equilíbrio
32×32	positivo	$\sim 1,22$ W	19,1%	alto speedup
64×64	negativo	3,99 W	76,1%	limite da plataforma

A configuração 32×32 aparece como um ponto especialmente interessante por combinar forte aceleração com margem de implementação ainda viável.

Limitações do Trabalho

Limitações identificadas

- Uso predominante de lógica geral em vez de DSPs dedicados;
- Maior dificuldade de timing para arranjos grandes;
- Ausência de otimizações avançadas de memória, como *tiling*;
- Avaliação focada em métricas de implementação, não em aplicações completas de IA.

As limitações não invalidam os resultados; elas indicam os próximos passos necessários para transformar a arquitetura em uma solução mais robusta e competitiva.

Lição principal

Não basta acelerar o núcleo

É preciso alimentar e escoar dados com eficiência

Conclusões do Trabalho

Resultados Alcançados

- Arquitetura sistólica parametrizável implementada e validada;
- Uso de DMA com barramentos HPC na Zynq;
- Análise experimental de latência, timing, recursos e potência;
- Comprovação de ganhos expressivos para matrizes maiores.

Speedup máximo

$>200\times$

Para a configuração 32×32 medida

Maior desafio

Escalabilidade física

Timing, Recursos e Potência em 64×64

Trabalhos Futuros

Melhorias arquiteturais

- Uso de blocos DSP;
- *Pipeline* a nível de Datapath;
- Hierarquia de memória com *buffering* e *tiling*;
- Múltiplos arranjos ou arranjos maiores.

Direções de Integração

- Integração com processadores RISC-V;
- Avaliação com redes neurais e workloads reais;
- Automação do mapeamento de algoritmos para o array.

Obrigado!